

Java Multithreading

20 Interview Questions and Answers with code examples

1 What is the difference between a process and a thread?

A **process** is an independent program in execution with its own memory space. A **thread** is a lightweight unit of execution inside a process. Threads of the same process share the heap and resources, but each thread has its own stack and program counter.

Creating a thread and switching between threads is cheaper than with processes, but shared memory means threads need proper synchronization to avoid data corruption.

2 What are the different ways to create a thread in Java?

There are four common ways: (1) extend the **Thread** class, (2) implement **Runnable**, (3) implement **Callable** and submit it to an **ExecutorService**, and (4) use a lambda expression with **Runnable**.

```
// 1. Extending Thread
class MyThread extends Thread {
    public void run() { System.out.println("Thread running"); }
}
new MyThread().start();

// 2. Runnable with a lambda (preferred)
Thread t = new Thread(() -> System.out.println("Runnable running"));
t.start();
```

3 Runnable vs Thread - which one should you prefer and why?

Runnable is preferred because:

- Java supports only single inheritance; extending **Thread** prevents extending any other class.
- **Runnable** separates the *task* from the *execution mechanism*, so the same task can run on a **Thread** or a thread pool.
- **Runnable** is a functional interface, so it works with lambda expressions.

4 Explain the lifecycle (states) of a thread.

A thread can be in one of six states defined in **Thread.State**:

- **NEW** - created but **start()** not yet called.
- **RUNNABLE** - running or ready to run, waiting for CPU time.
- **BLOCKED** - waiting to acquire a monitor lock (e.g., entering a synchronized block).
- **WAITING** - waiting indefinitely for another thread (**wait()**, **join()**, **park()**).
- **TIMED_WAITING** - waiting for a fixed time (**sleep(ms)**, **wait(ms)**, **join(ms)**).
- **TERMINATED** - **run()** has completed or thread died from an exception.

5 What is the difference between start() and run()?

start() creates a new thread of execution and the JVM then calls **run()** on that new thread. Calling **run()** directly just executes it like a normal method on the *current* thread - no new thread is created.

Calling **start()** twice on the same thread throws **IllegalThreadStateException**.

6 What is synchronization? What does the synchronized keyword do?

Synchronization controls access to shared resources so that only one thread at a time can execute a critical section. The **synchronized** keyword acquires the object's intrinsic lock (monitor) before entering the method or block and releases it on exit - even if an exception occurs. It gives both **mutual exclusion** and **memory visibility**.

```
class Counter {  
    private int count = 0;  
    public synchronized void increment() { count++; }  
    public synchronized int get() { return count; }  
}
```

7 What is the difference between object-level and class-level locks?

A synchronized **instance** method locks on **this** (the object), so different objects can run it concurrently. A synchronized **static** method locks on the **Class** object (e.g., **Counter.class**), so it is shared across all instances.

The two locks are independent - a thread holding the class lock does not block a thread that holds an object lock.

8 What is the volatile keyword? Does it make operations atomic?

volatile guarantees that reads and writes go to main memory (visibility) and prevents certain instruction reordering (happens-before). It does **not** provide atomicity, so compound operations such as **count++** on a volatile variable are still not thread-safe. It is ideal for simple status flags.

```
private volatile boolean running = true;  
public void stop() { running = false; } // immediately visible to others  
public void run() { while (running) { /* work */ } }
```

9 Explain wait(), notify() and notifyAll(). How is wait() different from sleep()?

These Object methods enable inter-thread communication. They must be called from inside a synchronized block. **wait()** releases the lock and suspends the thread until another thread calls **notify()** (wakes one thread) or **notifyAll()** (wakes all).

sleep() is a static Thread method that pauses the thread for a given time but does **not** release any lock. Always call wait() inside a *while* loop to guard against spurious wakeups.

```
synchronized (lock) {  
    while (queue.isEmpty()) {    // use a loop, not 'if'  
        lock.wait();           // releases the lock and waits  
    }  
    item = queue.remove();  
}  
// Producer: synchronized (lock) { queue.add(x); lock.notifyAll(); }
```

10 What is a deadlock? How can you prevent it?

A **deadlock** occurs when two or more threads wait forever for locks held by each other. It needs four conditions: mutual exclusion, hold-and-wait, no preemption, and circular wait.

The code below shows how it happens. **To prevent it:**

- Always acquire multiple locks in a consistent global order.
- Use tryLock() with a timeout instead of blocking forever.
- Avoid nested locks and keep critical sections small.
- Prefer higher-level utilities from java.util.concurrent.

```
// Thread 1 locks A then B  
synchronized (lockA) { synchronized (lockB) { /* work */ } }  
// Thread 2 locks B then A -> possible DEADLOCK  
synchronized (lockB) { synchronized (lockA) { /* work */ } }  
// Fix: every thread must acquire lockA first, then lockB.
```

11 What is a race condition and how do you make code thread-safe?

A **race condition** happens when the result depends on the unpredictable timing of threads accessing shared mutable data (e.g., two threads doing count++ at once and losing an update).

Ways to achieve thread safety: use synchronized or Lock, use atomic classes, use concurrent collections, use immutable objects, confine data to a single thread (ThreadLocal), or avoid shared state altogether.

12 What are join(), sleep() and yield()?

- **join()** - the calling thread waits until the target thread finishes (optionally with a timeout).
- **sleep(ms)** - the current thread pauses for the given time without releasing locks; may throw InterruptedException.
- **yield()** - a hint to the scheduler that the current thread is willing to give up the CPU; it is not guaranteed to have any effect.

13 What is a daemon thread?

A **daemon thread** is a low-priority background thread (e.g., the garbage collector) that does not prevent the JVM from exiting. When all user (non-daemon) threads finish, the JVM shuts down and daemon threads are terminated abruptly - so never use them for critical work like file writes. Call **setDaemon(true)** before start().

14 What is a thread pool and the ExecutorService framework? Why use it?

Creating a new thread for every task is expensive and unbounded. A **thread pool** reuses a fixed set of worker threads to run many tasks, improving performance and resource control. **ExecutorService** manages the pool and task lifecycle.

Common factory methods: `newFixedThreadPool(n)`, `newCachedThreadPool()`, `newSingleThreadExecutor()`, `newScheduledThreadPool(n)`. In production, creating a **ThreadPoolExecutor** directly gives control over queue size and rejection policy.

```
ExecutorService pool = Executors.newFixedThreadPool(4);
pool.submit(() -> System.out.println("Task by " +
    Thread.currentThread().getName()));
pool.shutdown(); // stop accepting new tasks; finish queued ones
```

15 What is the difference between Runnable and Callable? What is a Future?

Runnable: `run()` returns nothing and cannot throw checked exceptions. **Callable**: `call()` returns a result and can throw checked exceptions. A **Future** represents the pending result of an async task; `get()` blocks until it is ready, and it also supports `cancel()` and `isDone()`.

```
Callable<Integer> task = () -> 10 + 20;
Future<Integer> future = pool.submit(task);
int result = future.get(); // blocks until the result is ready
```

16 How does ReentrantLock differ from synchronized?

Both are reentrant (a thread can re-acquire a lock it already holds). **ReentrantLock** offers extra features:

- `tryLock()` with optional timeout, and `lockInterruptibly()`.
- Optional fairness policy (`new ReentrantLock(true)`) to favour the longest-waiting thread.
- Multiple Condition objects per lock (finer-grained wait/notify).
- Trade-off: you must release it manually in a finally block; `synchronized` releases automatically.

```
Lock lock = new ReentrantLock();
if (lock.tryLock(1, TimeUnit.SECONDS)) {
    try { /* critical section */ }
    finally { lock.unlock(); } // always unlock in finally
}
```

17 What are Atomic classes and how does CAS work?

Classes like **AtomicInteger**, **AtomicLong** and **AtomicReference** provide lock-free, thread-safe operations. They use **Compare-And-Swap (CAS)**, a CPU-level instruction: update the value only if it still equals the expected value, otherwise retry. This avoids blocking and is usually faster than locks under low-to-moderate contention.

```
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet();           // atomic, lock-free
counter.compareAndSet(1, 5);        // set to 5 only if current value is 1
```

18 ConcurrentHashMap vs Hashtable vs Collections.synchronizedMap - what is the difference?

- **Hashtable / synchronizedMap**: every method locks the entire map, so only one thread can work at a time - poor scalability.
- **ConcurrentHashMap**: uses CAS and fine-grained locking on individual bins (Java 8+), reads are mostly lock-free, so many threads can read and write concurrently.
- **ConcurrentHashMap** and **Hashtable** do not allow null keys or values (**HashMap** does).
- **ConcurrentHashMap** iterators are weakly consistent and do not throw **ConcurrentModificationException**.
- It also offers atomic operations such as **putIfAbsent()**, **compute()** and **merge()**.

19 What are CountdownLatch, CyclicBarrier and Semaphore?

These are synchronizers in `java.util.concurrent`:

- **CountDownLatch** - one or more threads wait until a set of operations completes; one-time use, cannot be reset.
- **CyclicBarrier** - a group of threads wait for each other at a barrier point; reusable.
- **Semaphore** - controls access to a resource using a number of permits (**acquire()/release()**); useful for rate limiting or connection pools.

```
CountDownLatch latch = new CountDownLatch(3);
// each worker thread calls latch.countDown() when it finishes
latch.await();    // main thread waits until the count reaches 0
```

20 What is ThreadLocal and when should you use it?

ThreadLocal gives every thread its own independent copy of a variable, so no synchronization is needed. Typical uses: per-thread user/session context, transaction IDs, and non-thread-safe helpers like **SimpleDateFormat**.

Caution: in thread pools, threads live long and get reused, so always call **remove()** when done, otherwise stale data or memory leaks can occur.

```
private static final ThreadLocal<SimpleDateFormat> fmt =
    ThreadLocal.withInitial(() -> new SimpleDateFormat("yyyy-MM-dd"));
// each thread sees its own copy; call fmt.remove() when finished
```

Bonus tip: Virtual Threads (Java 21+)

Interviewers increasingly ask about **virtual threads** (Project Loom). They are lightweight threads managed by the JVM rather than the OS, so you can run millions of blocking tasks cheaply. Create them with **`Executors.newVirtualThreadPerTaskExecutor()`** or **`Thread.ofVirtual().start(task)`**. They are best for I/O-bound work, not CPU-bound work.